UNIVERSIDAD INDUSTRIAL DE SANTANDER
FACULTAD DE INGENIERÍAS FISICOMECÁNICAS
ESCUELA DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

PLAN DE TRABAJO DE GRADO

FECHA DE PRESENTACIÓN: Bucaramanga, 09 de Abril de 2026

TÍTULO: Evaluación Experimental del Impacto Numérico y Energético de la Computación en Precisión Mixta mediante el uso de los Tensor Cores en GPUs NVIDIA para Kernels HPC

MODALIDAD: Trabajo de investigación

AUTOR:
Karen Dayana Mateus Gomez 2212765
William Andrés Urrutia Torres 2220058

DIRECTOR: Gilberto Javier Díaz Toro, Ph.D.

ENTIDAD INTERESADA: Escuela de Ingeniería de Sistemas e Informática

TABLA DE CONTENIDO

1 INTRODUCCIÓN 3
2 PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA 4
3 OBJETIVOS 7
3.1 OBJETIVO GENERAL 7
3.2 OBJETIVOS ESPECÍFICOS 7
4 MARCO DE REFERENCIA 8
4.1 Fundamentos teóricos 8
4.1.1 Formatos de representación de punto flotante 9
4.1.2 Arquitectura de Tensor Cores y Operaciones MAC 10
4.1.3 Propagación del Error Numérico y Deriva Acumulada 11
4.1.4 Compensación Numérica y Precisión Mixta 14
4.1.5 Termodinámica Computacional y Eficiencia Energética 14
4.2 Antecedentes del tema 15
4.2.1 La Resistencia de HPC a la Baja Precisión y el Potencial Energético 15
4.2.2 Tensor Cores y la Ruptura del Estándar IEEE 754 16
4.2.3 Algoritmos de Compensación y Casos de Éxito Computacional 18
5 METODOLOGÍA 18
5.1 Fase 1: Construcción de la Línea Base Analítica (FP64 y FP32) 19
5.2 Fase 2: Integración de Precisión Mixta y Activación de Tensor Cores 19
5.3 Fase 3: Cuantificación del Drift Numérico y Suma Compensada 19
5.4 Fase 4: Telemetría Energética, Diseño Factorial y Análisis Estadístico 20
5.5 Alcances y Limitaciones 20
5.5.1 Alcances 20
5.5.2 Limitaciones 21
6 CRONOGRAMA 22
7 PRESUPUESTO 23
8 BIBLIOGRAFÍA 25

EVALUACIÓN EXPERIMENTAL DEL IMPACTO NUMÉRICO Y ENERGÉTICO DE LA COMPUTACIÓN EN PRECISIÓN MIXTA MEDIANTE EL USO DE LOS TENSOR CORES EN GPUS NVIDIA PARA KERNELS HPC

1 INTRODUCCIÓN

Históricamente, la computación científica de alto rendimiento (HPC) ha dependido de manera casi exclusiva del estándar de punto flotante de doble precisión (FP64, definido por la norma IEEE 754) para garantizar la estabilidad numérica en simulaciones complejas. Sin embargo, el auge de la Inteligencia Artificial y el Aprendizaje Profundo (Deep Learning) ha transformado radicalmente la microarquitectura de los aceleradores gráficos (GPUs) modernos. En la actualidad, una porción masiva del silicio en las GPUs de NVIDIA está dedicada a unidades especializadas conocidas como Tensor Cores. Estas unidades están diseñadas para ejecutar operaciones de multiplicación y acumulación de matrices (MAC) a velocidades exorbitantes, pero utilizando formatos de menor precisión, tales como FP16 o Bfloat16 (BF16).

Esta divergencia arquitectónica plantea un dilema fundamental para la ingeniería de software científico: determinar si es posible aprovechar la masiva potencia de cálculo y la eficiencia energética de los Tensor Cores en algoritmos HPC tradicionales sin destruir la validez física o matemática de la simulación subyacente. Reducir la precisión de FP64 a formatos como FP32, FP16 o BF16 disminuye linealmente la presión sobre el ancho de banda de memoria y aumenta exponencialmente el caudal computacional (throughput). No obstante, este proceso inevitablemente introduce errores de truncamiento y redondeo que, al operar sobre algoritmos iterativos, pueden acumularse rápidamente hasta generar resultados divergentes.
Para abordar esta incógnita, el presente proyecto de investigación propone una evaluación exhaustiva y cuantitativa del paradigma conocido como Computación en Precisión Mixta. Mediante la implementación controlada de algoritmos canónicos en supercomputación, específicamente GEMM, Convoluciones y Stencils, se evaluará empíricamente el comportamiento del hardware al operar forzadamente en múltiples precisiones. El objetivo analítico es mapear el espacio de soluciones para encontrar el "Frente de Pareto" óptimo: aquellas configuraciones precisas donde la degradación numérica se mantiene en niveles estadísticamente aceptables para el dominio del problema, pero las ganancias en velocidad de ejecución y ahorro energético alcanzan su punto máximo.

2 PLANTEAMIENTO Y JUSTIFICACIÓN DEL PROBLEMA

En la era de la computación a exaescala, el paradigma de ejecutar por defecto todas las operaciones en doble precisión (FP64) se ha convertido en un cuello de botella cada vez menos sostenible, tanto desde la perspectiva del tiempo de ejecución como del consumo energético.Aunque FP64 ha sido históricamente el formato de referencia en computación científica por su capacidad para preservar la estabilidad numérica, la literatura reciente muestra que este conservadurismo puede limitar artificialmente el rendimiento de muchas aplicaciones científicas cuando existen alternativas de precisión mixta cuidadosamente controladas [1]. Al mismo tiempo, el aumento del costo energético de la supercomputación ha convertido la eficiencia en una restricción central del diseño y ejecución de aplicaciones HPC, por lo que ya no basta con resolver un problema correctamente: también es necesario hacerlo con un costo energético razonable [2].

En este contexto, las arquitecturas actuales de GPUs muestran una asimetría marcada entre el rendimiento en alta y baja precisión. En particular, los formatos reducidos, como FP16 y BF16, permiten aprovechar unidades especializadas capaces de ofrecer un rendimiento muy superior al de FP64, con un menor costo asociado al movimiento de datos y al consumo energético. Esta evolución arquitectónica se ha consolidado con la incorporación de Tensor Cores en las GPUs modernas de NVIDIA, los cuales se han convertido en uno de los mecanismos de aceleración más relevantes de las arquitecturas contemporáneas [3].

Como respuesta a estas limitaciones, las GPUs modernas han incorporado Tensor Cores, unidades especializadas para acelerar operaciones matriciales bajo esquemas de precisión mixta. Estos aceleradores permiten explotar formatos como FP16 y BF16 con un rendimiento significativamente superior al de FP64, por lo que se han convertido en un recurso clave para incrementar el throughput y mejorar el rendimiento por watt. Sin embargo, aprovechar esta capacidad no depende únicamente de la existencia del hardware, sino también de cómo se representan los operandos, cómo se efectúan las acumulaciones y cómo se aproxima numéricamente el producto matricial. Modelos recientes de Tensor Cores muestran precisamente que la ganancia de rendimiento y la exactitud final dependen de estas decisiones internas de implementación [4].

Desde una perspectiva arquitectónica, además, se han propuesto mecanismos orientados a reducir el consumo energético de los Tensor Cores, como el aprovechamiento de la sparsity y el apagado selectivo de cómputos inefectivos. Esto posiciona la precisión mixta no solo como una técnica de aceleración, sino también

como una estrategia de diseño orientada al equilibrio entre rendimiento, energía y calidad del resultado [5].

Aunque la precisión mixta puede mejorar la eficiencia computacional y energética en aplicaciones científicas, su adopción es limitada por preocupaciones de estabilidad y exactitud numérica. La reducción de precisión introduce errores que comprometen la confiabilidad de los resultados. Por lo tanto, el desafío es balancear el incremento de rendimiento con el mantenimiento de la exactitud requerida.

Este problema resulta especialmente relevante en kernels representativos de HPC como la multiplicación de matrices densas (GEMM), la convolución 2D y los operadores stencil 2D, los cuales constituyen patrones de cómputo fundamentales en simulación numérica, procesamiento científico de señales, métodos iterativos y resolución de ecuaciones diferenciales parciales. En el caso de GEMM, la reducción de precisión puede inducir errores acumulativos en productos y sumas repetidas, afectando la exactitud del resultado final. En convolución 2D, la baja precisión puede deteriorar la respuesta numérica debido a cancelaciones y a la sensibilidad del proceso de acumulación, especialmente cuando se emplean filtros con valores de distinta magnitud. En stencil 2D, entendido como un patrón regular de actualización local ampliamente utilizado en métodos de diferencias finitas, la situación es aún más crítica, ya que su formulación iterativa favorece la propagación y amplificación progresiva de pequeños errores de redondeo. Además, estudios recientes muestran que stencil y convolución comparten similitudes estructurales importantes y que incluso pueden transformarse en operaciones matriciales para ejecutarse sobre Tensor Cores, lo cual refuerza su pertinencia como objeto de estudio dentro de un mismo marco experimental [6].

Sin embargo, la posibilidad de adaptar convoluciones o stencils a operaciones matriciales no resuelve por sí sola el problema central. En la práctica, la aceleración observada en Tensor Cores no depende únicamente del algoritmo, sino también del modo en que el hardware implementa internamente la precisión mixta, la acumulación y la aproximación numérica del producto matricial [4]. Por tanto, no basta con demostrar que un kernel puede ejecutarse sobre Tensor Cores ni con constatar que el tiempo de ejecución disminuye. También es necesario verificar si esa aceleración sigue siendo científicamente útil al contrastarla con una referencia de mayor precisión y al analizarla conjuntamente con el consumo energético y la degradación del error numérico.

A pesar de los avances en precisión mixta y hardware tensorial, en HPC persiste una brecha científica y experimental importante. Aún no existen criterios cuantitativos suficientemente claros para determinar bajo qué condiciones es viable emplear

Tensor Cores en kernels científicos sin comprometer niveles aceptables de exactitud numérica. En particular, siguen siendo escasos los estudios que analicen de forma conjunta el tiempo de ejecución, el consumo energético y la degradación del error numérico en kernels representativos de HPC. Como consecuencia, los desarrolladores de software científico carecen todavía de una base experimental sólida para seleccionar la precisión más adecuada según el tipo de kernel y la tolerancia al error de la aplicación.

En consecuencia, esta investigación responde a una necesidad concreta de la computación científica contemporánea: desarrollar software energy-aware capaz de aprovechar el hardware moderno sin sacrificar la rigurosidad numérica exigida por la simulación científica. A partir de esta necesidad de establecer fronteras seguras de optimización entre rendimiento, consumo energético y exactitud numérica, se formula la siguiente pregunta de investigación:

¿En qué medida la implementación de esquemas de computación en precisión mixta, mediante el uso de Tensor Cores en GPUs NVIDIA, permite optimizar el rendimiento computacional y el consumo energético en kernels representativos de HPC, como GEMM, Convolución 2D y Stencil 2D, manteniendo el error numérico dentro de límites aceptables para la aplicación?

3 OBJETIVOS

3.1 OBJETIVO GENERAL

Determinar empíricamente las configuraciones de computación en precisión mixta con uso de Tensor Cores en GPUs NVIDIA que ofrezcan el mejor compromiso entre rendimiento computacional, consumo energético y exactitud numérica en kernels representativos de HPC, mediante una evaluación sistemática basada en mediciones de potencia, energía y métricas de error durante la ejecución, sin exceder niveles aceptables de error para la aplicación.

3.2 OBJETIVOS ESPECÍFICOS

1. Implementar y adaptar versiones paramétricas de kernels representativos de HPC (GEMM, Convolución 2D y Stencil 2D) en distintos formatos de precisión FP64, FP32, FP16 y BF16, activando explícitamente los Tensor Cores mediante el uso de bibliotecas oficiales (cuBLAS, cuDNN, CUTLASS) y kernels personalizados.
2. Medir y caracterizar el rendimiento computacional y el consumo energético de cada variante de precisión, cuantificando la latencia, el throughput efectivo (TFLOPS) y el Energy Delay Product (EDP), así como el consumo real de potencia y energía durante la ejecución mediante herramientas de perfilamiento y telemetría, como Nsight Compute, NVML para la GPU y RAPL para la CPU host.
3. Evaluar la degradación del error numérico y el drift acumulado en esquemas iterativos bajo distintas configuraciones de precisión, con y sin algoritmos de compensación (p. ej., suma de Kahan), utilizando métricas estándar (norma L2 y L_∞) y análisis estadístico (t-test, ANOVA), con el fin de establecer directrices que orienten la selección óptima de precisión según la tolerancia de la aplicación.
4. Realizar un análisis estadístico multivariado sobre el espacio tridimensional conformado por Tiempo, Energía y Error, estableciendo el Frente de Pareto para extraer directrices de ingeniería que determinen la estrategia de precisión óptima según la tolerancia de la aplicación objetivo.

4 MARCO DE REFERENCIA

4.1 Fundamentos teóricos

La evaluación integral del compromiso entre el rendimiento computacional, el consumo energético y la exactitud numérica en el ámbito de la computación de alto rendimiento (HPC) es una tarea multidimensional que exige la consideración simultánea de varios factores interdependientes. Específicamente, esta evaluación debe tener en cuenta la representación numérica utilizada para codificar los datos (por ejemplo, precisión simple, doble, o formatos mixtos/reducidos), la arquitectura intrínseca del acelerador empleado (como CPUs o GPUs con unidades especializadas como los Tensor Cores), y la manera en que los errores inherentes a las operaciones de punto flotante se propagan y acumulan a lo largo de las etapas del algoritmo numérico.

Dentro de este marco, los esquemas de precisión mixta emergen como un paradigma particularmente relevante y complejo. El objetivo principal de estos esquemas es mejorar el rendimiento y la eficiencia energética mediante la utilización de formatos de menor precisión (como FP16, BFloat16 o INT8) en las partes del cálculo donde el impacto en la exactitud es mínimo o controlable. Sin embargo, el incremento del rendimiento no es una consecuencia directa y automática de la simple adopción de formatos de baja precisión. Por el contrario, depende fundamentalmente de una distribución estratégica y cuidadosa de las precisiones a lo largo de todo el algoritmo, lo que implica determinar qué operaciones pueden ejecutarse con menor precisión sin comprometer la convergencia, la estabilidad o la calidad del resultado final [7]. La efectividad de la precisión mixta está intrínsecamente ligada a la capacidad de gestionar la propagación del error y de garantizar que la reducción de precisión no introduzca inestabilidades numéricas críticas.

Adicionalmente, cuando se emplean arquitecturas de aceleración modernas, como las Unidades de Procesamiento Gráfico (GPUs), el comportamiento numérico efectivo se complica debido a las características específicas de las unidades de cálculo especializadas. En el caso de las GPUs contemporáneas, los Tensor Cores son unidades clave diseñadas para acelerar operaciones densas como la multiplicación de matrices. El comportamiento numérico preciso de estos Tensor Cores está condicionado por detalles de implementación que a menudo se desvían de un modelo idealizado de aritmética de punto flotante IEEE 754. Estos detalles incluyen el modo de redondeo específico utilizado (por ejemplo, hacia el más cercano, hacia cero, etc.), el tratamiento que se da a los números subnormales (o denormalizados), y el orden exacto en que se realiza la acumulación de los productos intermedios. Estos aspectos internos pueden llevar a que el resultado numérico difiera sutilmente, pero potencialmente de manera significativa en ciertos

contextos algorítmicos, del comportamiento que se esperaría bajo una estricta implementación estándar IEEE [8]. Por lo tanto, la optimización para HPC en estas arquitecturas requiere una comprensión profunda de estas idiosincrasias de hardware, y no solo de los principios algorítmicos generales.

4.1.1 Formatos de representación de punto flotante

En la computación científica, los números reales se aproximan mediante el estándar IEEE 754. Matemáticamente, el valor de un número de punto flotante puede expresarse como:

$$V = (-1)^{s} \times (1 + m) \times 2^{e-bias}$$

Donde s representa el bit de signo, m es la mantisa o fracción, y e es el exponente sesgado. Tradicionalmente, la computación de alto rendimiento ha dependido del formato FP64 (doble precisión),el cual dispone de 64 bits, con 11 bits de exponente y 52 de mantisa para garantizar que el error de redondeo, conocido como épsilon de la máquina ($\epsilon \approx 2.22 \times 10^{-16}$), sea matemáticamente despreciable. Para acelerar el cómputo, el hardware moderno ha incorporado formatos de menor precisión, como el FP32 (precisión simple), que utiliza 32 bits y reduce la mantisa a 23 bits ($\epsilon \approx 1.19 \times 10^{-7}$).

El ámbito de la Computación de Alto Rendimiento (HPC) ha dependido tradicionalmente del formato de punto flotante de doble precisión (FP64) como estándar fundamental. Esta preferencia histórica se debe a la elevada exactitud y el amplio rango dinámico que ofrece FP64, características cruciales para garantizar la fiabilidad y la convergencia de simulaciones científicas complejas y cálculos numéricos de alta fidelidad. Sin embargo, la evolución continua de las arquitecturas de hardware, impulsada por la búsqueda de mayor eficiencia energética y rendimiento, ha introducido soporte nativo para formatos de precisión reducida, incluyendo FP16 (media precisión) y BF16 (formato Brain Floating Point). Estos formatos de menor precisión no solo se integran de forma natural en los modernos esquemas de precisión mixta, sino que también representan una estrategia clave para optimizar el rendimiento.

El creciente interés en estas representaciones de baja precisión se justifica por su capacidad para acelerar significativamente operaciones computacionales intensivas, siendo un ejemplo paradigmático las operaciones de multiplicación-acumulación de matrices (matrix multiply–accumulate). Esta ganancia en velocidad se debe, en gran medida, a que la reducción de precisión disminuye drásticamente los requisitos de ancho de banda y el costo de almacenamiento y transferencia de datos. Además, el

hardware moderno ha incorporado unidades de procesamiento especializadas, como los Tensor Cores en las GPUs de NVIDIA, que están diseñadas específicamente para maximizar el rendimiento al trabajar con operandos de baja precisión.

No obstante, esta mejora en el rendimiento viene acompañada de importantes desafíos numéricos. La reducción del número de bits dedicados a la mantisa y al exponente incrementa intrínsecamente la sensibilidad del cálculo a los errores de redondeo, la pérdida de significancia, y el fenómeno de la cancelación numérica. Estos problemas se exacerban particularmente cuando la acumulación en precisión mixta se implementa mediante unidades matriciales especializadas como los Tensor Cores, donde el control sobre la precisión intermedia de la acumulación puede ser menos granular que en las unidades de punto flotante estándar (FPUs).

En esencia, la causa fundamental de estos efectos es un compromiso inherente: la disminución de la precisión facilita el uso de hardware más eficiente y reduce el movimiento de datos, pero simultáneamente incrementa la susceptibilidad a las imprecisiones numéricas. Por lo tanto, la selección del formato de punto flotante adecuado en un entorno de HPC o Deep Learning trasciende la mera consideración del rendimiento (throughput). Se convierte en una decisión crítica que exige un análisis riguroso de las propiedades numéricas del algoritmo implementado y las tolerancias a errores del problema específico. El diseño algorítmico debe, por necesidad, incorporar estrategias de mitigación de errores para asegurar que la ganancia de velocidad no comprometa la exactitud requerida para la validez de los resultados.

4.1.2 Arquitectura de Tensor Cores y Operaciones MAC

La base arquitectónica de la precisión mixta en GPUs NVIDIA son los Tensor Cores, unidades especializadas para operaciones de multiplicación y acumulación matricial. Su operación fundamental puede representarse como

$$D = A \times B + C$$

donde A y B son las matrices de entrada, C es la matriz acumuladora y D es la matriz resultante. Estudios sobre el comportamiento numérico de Tensor Cores, esta operación se toma explícitamente como punto de partida para analizar cómo se implementa la aritmética interna en arquitecturas Volta, Turing y Ampere [8].

Una propiedad central y definitoria de estas unidades especializadas, conocidas como Tensor Cores, es su capacidad intrínseca para funcionar como potentes aceleradores de la multiplicación matricial, empleando un esquema de precisión

mixta. Esta arquitectura híbrida está diseñada meticulosamente para optimizar la eficiencia computacional sin sacrificar indebidamente la exactitud numérica.

En el contexto de los Tensor Cores que han sido objeto de análisis y experimentación, se observa que el diseño permite que los datos de entrada se almacenen en formatos de precisión reducida, como FP16 o incluso formatos de precisión aún menor. No obstante, la etapa crítica de acumulación de los productos resultantes se lleva a cabo en una precisión significativamente mayor, típicamente FP32 (punto flotante de precisión simple). El propósito fundamental de esta estrategia de precisión mixta es mitigar de manera efectiva la propagación del error de cuantización, conteniendo así, al menos parcialmente, la inherente pérdida de exactitud que podría surgir al emplear exclusivamente formatos de menor precisión durante todo el proceso computacional [8].

La relevancia y versatilidad de esta organización arquitectónica trascienden el dominio del aprendizaje profundo (Deep Learning). De hecho, esta misma filosofía de aceleración se ha extendido exitosamente a aplicaciones científicas y de ingeniería mucho más avanzadas y estructuradas. Como ejemplo paradigmático, Baboulin y sus colaboradores han demostrado la explotación eficiente de los Tensor Cores que operan en el modo FP16/FP32 para acelerar de forma significativa los productos de matrices densas de propósito general (GEMM, General Matrix Multiply). Estos productos GEMM constituyen la base algorítmica de los métodos de aproximación de bajo rango, los cuales son esenciales en el procesamiento de datos a gran escala y la reducción dimensional. Este logro confirma categóricamente que estas unidades de procesamiento no están confinadas únicamente a la aceleración de modelos de redes neuronales, sino que también representan una herramienta invaluable para la aceleración de una amplia gama de algoritmos numéricos estructurados de alto rendimiento [10]. La incorporación de los Tensor Cores en estos campos abre nuevas vías para resolver problemas de gran escala con mayor rapidez y eficiencia energética.

4.1.3 Propagación del Error Numérico y Deriva Acumulada

La reducción de la precisión en las representaciones numéricas no es trivial; inherentemente introduce errores de redondeo (al ajustar un número a la representación más cercana) y de truncamiento (al descartar los dígitos menos significativos) en cada operación aritmética elemental. Estos errores iniciales, aunque individualmente pequeños, son locales.

Sin embargo, el verdadero desafío surge en los cálculos iterativos, extensos o que implican una gran cantidad de operaciones sucesivas. En estos escenarios, los errores locales dejan de ser aislados y comienzan a propagarse y acumularse a

través del sistema. Esta propagación de errores puede llevar a que la solución final se desvíe significativamente del resultado matemáticamente exacto, incluso si los algoritmos utilizados son robustos desde una perspectiva teórica.

Por lo tanto, la gestión y el control del error se convierten en un aspecto crucial y complejo al trabajar con precisión reducida. Es imperativo no solo estimar el error local introducido por cada operación, sino también modelar y mitigar cómo este error se amplifica a lo largo de las etapas del cálculo.Esto requiere la implementación y el dominio de técnicas especializadas en análisis numérico. El propósito fundamental de estas técnicas es la de asegurar que la precisión computacionalmente viable sea suficiente para mantener la fiabilidad y la validez de los resultados obtenidos en el proceso de cálculo. En el contexto de simulaciones o modelos complejos, la elección adecuada del esquema numérico, la discretización y el manejo de errores de redondeo y truncamiento se vuelven aspectos críticos para la integridad de la solución.

En este sentido, las investigaciones han arrojado luz sobre la naturaleza multifacética del error en las operaciones numéricas. Se ha demostrado de manera concluyente que el error inherente no solo es una función directa del formato de precisión empleado (como la simple o doble precisión en punto flotante, o formatos personalizados), sino que también está íntimamente ligado a factores intrínsecos de la operación y de los datos de entrada. Específicamente, se ha identificado que el número de condición del producto interno es un indicador clave de la sensibilidad de la operación a pequeñas perturbaciones en los datos o a errores de precisión. Un número de condición alto implica que el problema es intrínsecamente mal condicionado, amplificando cualquier error de precisión inicial. Además, la sensibilidad de la operación aplicada a su salida también juega un papel determinante, refiriéndose a cómo las variaciones en la entrada o los errores intermedios se propagan y afectan el resultado final. Esta complejidad subraya la necesidad de un análisis riguroso de la propagación de errores y de la estabilidad numérica de los algoritmos utilizados, como lo documentan estudios seminales en el área [11]. Este trabajo propone una formulación para la elección de precisión basada en un análisis componente a componente del error, sugiriendo que ciertos productos internos pueden calcularse con baja precisión, mientras que otros requieren una recomputación con una precisión superior para mantener la exactitud del resultado global.

Para cuantificar rigurosamente esta degradación, el proyecto utilizará dos métricas fundamentales sobre el dominio de resultados. Sea X la matriz resultante en FP64 (considerada el Ground Truth) y $\hat{X}$ la matriz computada en precisión reducida, se

define la Norma L_2 (Error Cuadrático Medio Euclidiano) para medir la desviación global promedio de toda la simulación:

$$L_2 = \sqrt{\sum_{i=1}^{N} \left(X_i - \hat{X}_i\right)^2}$$

Asimismo, para detectar inestabilidades puntuales o picos de divergencia matemática inaceptables, se emplea la Norma L_∞ (Error Máximo Absoluto):

$$L_\infty = \left|X_i - \hat{X}_i\right|$$

Para mitigar la divergencia de precisión sin sacrificar el rendimiento superior que ofrecen las representaciones numéricas de baja precisión (como FP16 o BF16), se propone una evaluación exhaustiva de la inserción algorítmica de la Suma Compensada de Kahan. Este método, una técnica matemática avanzada para la reducción de errores de redondeo, opera capturando el error de redondeo generado en cada operación de adición individual y almacenándolo de manera explícita en una variable auxiliar, denominada "compensación" o "error acumulado". Subsiguientemente, este error acumulado es reintegrado y compensado en el siguiente paso temporal o iteración.

La implementación de la Suma Compensada de Kahan ofrece una ventaja teórica significativa: permite lograr una precisión efectiva muy cercana a la obtenida con formatos de punto flotante de 32 bits (FP32) —o incluso superior en ciertos escenarios—, mientras que el hardware subyacente continúa ejecutando las operaciones a la velocidad inherente a los registros de baja precisión (FP16/BF16). Esta técnica representa, por lo tanto, un puente crítico entre la necesidad de alto rendimiento computacional y la exigencia de una robustez numérica adecuada en simulaciones o modelos complejos.

Por otro lado, en el ámbito específico de las operaciones matriciales aceleradas (como la multiplicación de matrices, o GEMM) que son fundamentales en el Deep Learning y la computación de alto rendimiento, la propagación del error es un fenómeno que se encuentra intrínsecamente ligado a las características de la precisión mixta (FP16 o BF16). Este fenómeno está profundamente influenciado por dos factores clave de la arquitectura hardware:

1. La Implementación de la Operación Fused Multiply-Add (FMA): La FMA, que combina la multiplicación y la adición en una única instrucción, es crucial. Si bien la FMA está diseñada para minimizar un error de redondeo intermedio, su precisión final y la manera en que maneja la mantisa y el exponente en formatos de baja precisión impactan directamente la acumulación del error.
2. El Mecanismo Interno de Acumulación (Accumulator): Los aceleradores modernos (GPUs, TPUs, etc.) a menudo utilizan acumuladores internos que operan con una precisión superior a la de los operandos de entrada (e.g., acumuladores FP32 o incluso FP64 para operandos FP16). La elección y el mecanismo efectivo de este acumulador de mayor precisión (conocido como upcasting o high-precision accumulation) es el factor determinante en la precisión final del resultado de una operación matricial de gran escala.

Consecuentemente, cualquier análisis riguroso y significativo de la precisión del error en sistemas que emplean precisión mixta no puede limitarse a la simple evaluación del formato de entrada. Requiere, imperativamente, una consideración explícita, detallada y experimental del mecanismo efectivo de acumulación que está siendo empleado por el hardware acelerador específico en cuestión [8]. Ignorar esta consideración arquitectónica llevaría a conclusiones erróneas sobre la robustez numérica del sistema.

4.1.4 Compensación Numérica y Precisión Mixta

Dado que la utilización de formatos de baja precisión, como FP16 o TF32, introduce inherentemente errores de redondeo que pueden acumularse y comprometer la exactitud del resultado final, especialmente en algoritmos iterativos o con alta dependencia de la precisión intermedia, una estrategia de mitigación fundamental consiste en la combinación sinérgica de la aceleración proporcionada por el hardware especializado (como los Tensor Cores en GPUs modernas) con mecanismos algorítmicos de corrección de errores.

En este contexto, la precisión mixta (Mixed Precision) emerge como un paradigma sofisticado. No debe entenderse, como se indicó, como una simple sustitución global y uniforme de la precisión estándar (e.g., FP64 o FP32) por un formato de baja precisión. Por el contrario, se ha demostrado que la clave reside en la asignación controlada y no uniforme de la precisión. Estudios en la literatura [11] han revelado que una estrategia de acumulación guiada por la sensibilidad de cada componente o subcálculo del algoritmo (i.e., identificando qué partes son más susceptibles a la degradación numérica) puede mejorar significativamente el equilibrio entre el costo computacional (al maximizar el uso de formatos de baja precisión y la aceleración por hardware) y la exactitud del resultado final, superando el rendimiento de una política

de precisión uniforme. Esto establece que la optimización no es binaria, sino que requiere un análisis detallado de la propagación del error numérico dentro de la estructura del algoritmo.

Complementariamente a la asignación no uniforme, el refinamiento iterativo en precisión mixta se presenta como una herramienta poderosa para garantizar la exactitud. Este método [9] aprovecha la capacidad de aceleración de los Tensor Cores (o unidades de procesamiento equivalentes) para realizar la mayor parte de las operaciones computacionales intensivas (como la factorización o la resolución de sistemas lineales) en baja precisión. Sin embargo, la exactitud no se sacrifica, ya que el proceso incorpora sistemáticamente correcciones que se calculan en una precisión superior (e.g., FP64 o FP32). Estas correcciones, basadas típicamente en el cálculo del residuo y su posterior refinamiento, compensan la degradación numérica introducida por la baja precisión en los pasos de cómputo principales.

Esta aproximación resalta un principio crucial: la estabilidad y exactitud final de un cálculo en precisión mixta no dependen únicamente de la elección del formato de cómputo inicial para las operaciones principales. Más bien, la robustez del sistema se sustenta en la incorporación deliberada de mecanismos de compensación o refinamiento. Estos mecanismos son esenciales para corregir la degradación numérica inherente a la baja precisión, asegurando que, a pesar de la aceleración por hardware, se cumplan los requisitos de exactitud necesarios para la convergencia del algoritmo o la validez física del resultado.

4.1.5 Termodinámica Computacional y Eficiencia Energética

La motivación de la precisión mixta no es exclusivamente temporal, sino también energética. Desde una perspectiva arquitectónica, Hafezan y Atoofian proponen un diseño de Tensor Cores con modos exactos y aproximados, orientado explícitamente a reducir el consumo de energía en aplicaciones tolerantes a determinadas inexactitudes.

En su estudio, la combinación de unidades exactas y aproximadas permite obtener ahorros energéticos significativos, manteniendo la precisión requerida por los modelos evaluados [4]. Para estudiar el equilibrio entre energía y tiempo suele emplearse la métrica Energy-Delay Product (EDP),

$$EDP = E_{total} \times T_{exec}$$

Donde E_{total} representa el consumo total en Joules (medido a través de las API NVML y RAPL) y T_{exec} el tiempo de ejecución de la aplicación en segundos. El éxito

científico de una variante de precisión mixta se definirá identificando el punto exacto en el espacio tridimensional (Rendimiento, Energía, Error) donde se logra minimizar el EDP, manteniendo estrictamente las normas L_2 y L_∞ por debajo del límite de tolerancia preestablecido por la física del problema modelado.

4.2 Antecedentes del tema

4.2.1 La Resistencia de HPC a la Baja Precisión y el Potencial Energético

En computación de alto rendimiento, la adopción de formatos de baja precisión ha sido históricamente limitada debido a la necesidad de garantizar estabilidad numérica, reproducibilidad y confiabilidad en aplicaciones científicas. A diferencia de otros dominios de cómputo, en HPC la reducción de precisión no puede asumirse únicamente como una estrategia de optimización del rendimiento, ya que sus efectos se reflejan directamente en la calidad del resultado final. En este sentido, la cautela de la comunidad no responde solo a una preferencia tradicional por FP64, sino al hecho de que disminuir la precisión puede aumentar los errores de redondeo, favorecer la acumulación de desviaciones numéricas y, además, incrementar la sensibilidad de las aplicaciones frente a errores silenciosos en plataformas de gran escala [12].

Sin embargo, esta postura ha comenzado a cambiar a medida que las restricciones de rendimiento y consumo energético se han vuelto más críticas. El crecimiento de arquitecturas aceleradas con soporte para operaciones en precisión reducida ha puesto de manifiesto que la precisión mixta puede ofrecer ventajas importantes cuando se aplica de manera controlada. En particular, el uso de Tensor Cores ha mostrado que es posible acelerar operaciones de alto costo computacional mediante formatos de baja precisión y conservar la exactitud final cuando se incorporan mecanismos de corrección adecuados. Esta idea se evidencia en solucionadores que utilizan refinamiento iterativo, donde la parte costosa del cálculo se ejecuta en baja precisión, mientras que la corrección del error residual se realiza en una precisión superior [13].

De forma complementaria, estudios recientes han mostrado que estos beneficios no se limitan a kernels aislados, sino que pueden extenderse a aplicaciones científicas más cercanas a escenarios reales de HPC. En códigos basados en métodos espectrales, por ejemplo, se ha observado que una reducción selectiva de precisión puede disminuir simultáneamente el tiempo de solución y el consumo energético, manteniendo en determinados casos una exactitud comparable a la obtenida en doble precisión [14]. Este tipo de resultados resulta especialmente relevante porque

desplaza la discusión desde la viabilidad teórica de la precisión mixta hacia su utilidad práctica dentro de aplicaciones científicas concretas.

En conjunto, estos antecedentes indican que la discusión actual en HPC no consiste en decidir entre conservar de forma estricta FP64 o sustituirlo indiscriminadamente por formatos reducidos. Más bien, el problema se centra en establecer bajo qué condiciones la precisión mixta puede incorporarse de manera segura, de forma que los beneficios en rendimiento y energía no comprometan la estabilidad numérica ni la confiabilidad de los resultados. Desde esta perspectiva, la resistencia histórica de HPC a la baja precisión no debe entenderse como un obstáculo, sino como el punto de partida para definir criterios de uso más rigurosos y mejor fundamentados.

4.2.2 Tensor Cores y la Ruptura del Estándar IEEE 754

La aparición de los Tensor Cores ha introducido un cambio relevante en la discusión contemporánea sobre aceleración numérica en GPU, al ofrecer unidades especializadas para operaciones de multiplicación y acumulación matricial bajo esquemas de precisión mixta. No obstante, su incorporación al ámbito de la computación científica no puede analizarse únicamente desde la óptica del incremento de rendimiento. En contextos HPC, donde la validez del resultado depende de la estabilidad aritmética del cómputo, resulta indispensable comprender con detalle el comportamiento numérico real de estas unidades. En este sentido, estudios experimentales que han empleado los Tensor Cores en arquitecturas Volta, Turing y Ampere han evidenciado que aspectos críticos como la precisión de los resultados intermedios, el tratamiento de subnormales, el modo de redondeo, el orden de acumulación y la normalización de sumas parciales no se encuentran completamente especificados por la propia documentación de NVIDIA [15]. Esta observación es clave, pues mueve la discusión de lo arquitectónico a la reproducibilidad numérica y el control del error en hardware especializado.

Estudios recientes indican que la implementación de Tensor Cores no asegura una mejora exhaustiva de la robustez computacional. Estos analizan el impacto de las unidades y la precisión mixta en la confiabilidad de la multiplicación de matrices en GPU. Sus hallazgos indican que las operaciones de baja precisión pueden aumentar la cantidad de datos correctos, aunque también incrementan la severidad de los errores cuando estos se presentan. [16]. En otras palabras, la aceleración mediante el uso de hardware tensorial podría aumentar la sensibilidad a perturbaciones numéricas o fallos. Esto es clave en aplicaciones científicas y de ingeniería, donde un resultado correcto exige no solo completar el cálculo, sino también mantener la fidelidad cuantitativa respecto a una referencia de alta precisión.

Ante la disyuntiva entre rendimiento y exactitud, se han propuesto estrategias algorítmicas con el objetivo de aprovechar los Tensor Cores sin incurrir directamente en la degradación inherente a FP16 o BF16. Para lograr esto, trabajos recientes examinaron la multiplicación matricial en aritmética multiword sobre Tensor Cores y demostraron que esquemas como double-fp16 pueden mejorar el compromiso entre desempeño y precisión. No obstante, en la práctica, estos esquemas no siempre logran la exactitud de fp32 [17]. Sus resultados atribuyen esta discrepancia a características internas de redondeo y acumulación, lo cual corrobora que el análisis de la precisión mixta debe considerar no solo el formato numérico empleado, sino también la semántica aritmética efectiva del hardware.

La dimensión energética es clave. Varias propuestas sugieren una arquitectura de precisión mixta para reducir el consumo de energía de los Tensor Cores. Esto se logra mediante la combinación de componentes exactos y aproximados, reportando ahorros significativos en cargas de trabajo de redes neuronales sin una pérdida crítica de exactitud [18]. Aunque validado principalmente en Redes Neuronales Profundas (DNNs), este enfoque subraya que la interdependencia entre precisión, energía y calidad del resultado es un aspecto de diseño fundamental al utilizar estas unidades.

De manera complementaria, se ha demostrado que los Tensor Cores también pueden incorporarse en algoritmos científicos más allá del aprendizaje profundo, mediante estrategias híbridas que combinan baja precisión con refinamiento iterativo [20]. Sus resultados indican que es posible obtener aceleraciones importantes y, al mismo tiempo, recuperar parte de la exactitud perdida mediante mecanismos de corrección. En conjunto, estos trabajos evidencian que el verdadero desafío ya no consiste solo en demostrar que los Tensor Cores aceleran el cálculo, sino en establecer bajo qué condiciones esa aceleración sigue siendo numérica y energéticamente viable para aplicaciones científicas exigentes. Esa es precisamente la necesidad que da soporte a una evaluación experimental del compromiso entre rendimiento, consumo energético y error numérico en kernels HPC representativos.

4.2.3 Algoritmos de Compensación y Casos de Éxito Computacional

La utilidad de la precisión mixta en computación científica no depende únicamente del hardware disponible, sino también de la incorporación de mecanismos algorítmicos capaces de compensar la pérdida de exactitud introducida por los formatos reducidos. En este sentido, la literatura reciente coincide en que la adopción de precisión mixta resulta más efectiva cuando se combina con estrategias de corrección y refinamiento que permitan conservar la estabilidad numérica del cálculo [1].

Entre estos enfoques, el refinamiento iterativo en precisión mixta se ha consolidado como una de las estrategias más representativas. Su principio consiste en ejecutar en baja precisión las operaciones de mayor costo computacional y reservar una precisión superior para la corrección del error residual. De esta manera, es posible aprovechar la aceleración del hardware especializado sin renunciar a la exactitud final requerida por la aplicación. Este esquema ha mostrado resultados favorables en la solución de sistemas lineales acelerados con Tensor Cores, donde la ganancia en rendimiento se combina con mecanismos de corrección capaces de preservar la calidad numérica de la solución [13].

De forma complementaria, la viabilidad de estas estrategias no se limita al álgebra lineal densa, sino que también se ha extendido a aplicaciones científicas más cercanas a escenarios reales de HPC. En códigos basados en elementos espectrales, por ejemplo, la reducción selectiva de precisión ha permitido disminuir simultáneamente el tiempo de solución y el consumo energético, manteniendo en determinados escenarios una exactitud comparable a la obtenida en doble precisión [14]. En conjunto, estos antecedentes indican que los algoritmos de compensación constituyen un elemento fundamental para que la precisión mixta pueda adoptarse de manera confiable y efectiva en entornos HPC.

5 METODOLOGÍA

La presente investigación se enmarca en un diseño experimental cuantitativo, con enfoque empírico orientado a la evaluación del compromiso entre rendimiento computacional, consumo energético y exactitud numérica en esquemas de precisión mixta sobre GPU.El diseño metodológico busca aislar el efecto de la reducción de precisión sobre el desempeño, la eficiencia energética y la estabilidad numérica de kernels representativos de HPC. Para garantizar la reproducibilidad de los experimentos y reducir la variabilidad del entorno de ejecución, el desarrollo se organizará en cuatro fases.

5.1 Fase 1: Construcción de la Línea Base Analítica (FP64 y FP32)

El objetivo de esta fase es desarrollar las implementaciones de referencia que servirán como base de comparación para las ejecuciones en precisión reducida.

- **Programación de Referencia:** Se implementarán tres familias de kernels: Multiplicación de Matrices Densas (GEMM), Convolución 2D y operadores de Stencil (2D), operando estrictamente bajo el estándar IEEE 754 en doble precisión (FP64) y precisión simple (FP32).

- **Integración de Bibliotecas:** Para establecer una línea base de alto desempeño, se utilizarán bibliotecas oficiales de NVIDIA, principalmente cuBLAS y cuDNN, en las operaciones de álgebra lineal y convolución.
- **Perfilamiento Nominal:** Se obtendrán las métricas base de tiempo de ejecución, latencia y throughput aritmético teórico, sin recurrir aún a las unidades Tensor Cores.

5.2 Fase 2: Integración de Precisión Mixta y Activación de Tensor Cores

Esta fase es el componente central de la ingeniería de bajo nivel. En ella se abordará la adaptación de los algoritmos y la validación del hardware, por medio de la adaptación de los kernels a formatos reducidos y en la validación del uso efectivo de Tensor Cores.

- **Refactorización a Baja Precisión:** Se inyectarán los tipos de datos de 16 bits (FP16 y BF16), forzando al compilador (nvcc) a enrutar el cálculo hacia los Tensor Cores.
- **Adaptación de Estructuras y Kernels:** En GEMM se emplearán extensiones de precisión mixta y plantillas CUTLASS, prestando atención a la alineación y coalescencia de memoria. En el caso de stencil 2D, se desarrollarán kernels personalizados adaptados a la estructura iterativa del vecindario.
- **Validación de rendimiento:** se utilizará NVIDIA Nsight Compute para verificar la activación efectiva de Tensor Cores y caracterizar el comportamiento en términos de latencia, ocupación y throughput.

5.3 Fase 3: Cuantificación del Drift Numérico y Suma Compensada

Una vez habilitadas las ejecuciones en precisión reducida, esta fase se orienta a medir la degradación numérica respecto a la solución de referencia.

- **Cálculo de Desviaciones:** Se contrastarán las matrices resultantes de las ejecuciones en baja precisión contra la solución Ground Truth (FP64).
- **Mapeo del Drift:** En los algoritmos iterativos, se registrará el error acumulado a lo largo de los pasos temporales, calculando sistemáticamente la Norma L_2 para medir la desviación global y la Norma L_∞para identificar el error máximo.
- **Mitigación Algorítmica:** Se instrumentarán los kernels de Stencil con el algoritmo de compensación de Kahan, evaluando si retener el error en una variable auxiliar mitiga la pérdida de exactitud sin incurrir en un overhead de tiempo que anule las ganancias obtenidas.

5.4 Fase 4: Telemetría Energética, Diseño Factorial y Análisis Estadístico

La fase final del proyecto culmina con la consolidación de los objetivos específicos segundo y cuarto, mediante la integración del modelado termodinámico y la construcción del espacio tridimensional de soluciones.

- **Aislamiento del Entorno Termodinámico:** Se fijarán las frecuencias de reloj de la GPU con el fin de mitigar el ruido térmico y asegurar la naturaleza determinista del cálculo de consumo de potencia.
- **Medición de Energía Automatizada:** Se ejecutará un barrido paramétrico que abarcará variaciones en las dimensiones de las matrices (desde 512x512 hasta 4096x4096) y en los dominios 2D/3D. Durante esta ejecución, se capturará el consumo físico de energía, expresado en Joules, tanto de la placa base como de la GPU, utilizando para ello las interfaces RAPL y NVML.
- **Análisis estadístico:** se aplicarán pruebas de inferencia estadística, como t-test y ANOVA, para determinar la significancia de las diferencias observadas entre configuraciones.
- **Modelado del Frente de Pareto:** Los resultados obtenidos permitirán la proyección del espacio de soluciones en función de Tiempo, Energía y Error, lo que a su vez derivará en la formulación de directrices concretas para la arquitectura de software.

5.5 Alcances y Limitaciones

5.5.1 Alcances

El alcance de esta investigación se establece en torno a la evaluación del hardware de aceleración de precisión reducida en un entorno experimental rigurosamente controlado. El desarrollo y la fase de experimentación se limitarán exclusivamente a la ejecución intra-nodo sobre una plataforma de cómputo heterogénea, la cual estará compuesta por una CPU Intel anfitriona y una GPU NVIDIA de arquitectura reciente con soporte para Tensor Cores.

Como casos de estudio instrumentales, se implementarán tres familias de kernels críticos para la computación científica. La evaluación comprenderá la multiplicación de matrices densas (GEMM) en dimensiones que varían desde 512x512 hasta 4096x4096 elementos. Adicionalmente, se incorporarán operaciones de Convolución 2D, analizando diversos tamaños de filtro y lotes (batch), junto con operadores de diferencias finitas (Stencil) sobre dominios espaciales de 512^2 a 2048^2 en 2D.

El análisis numérico evaluará exclusivamente cuatro formatos de representación de punto flotante dictados por la arquitectura del hardware: Doble Precisión (FP64), Precisión Simple (FP32), Media Precisión (FP16) y Brain Floating Point (BF16). Para la implementación de las rutinas algebraicas y la activación de las unidades tensoriales, el desarrollo de software se apoyará estrictamente en el ecosistema de

NVIDIA. Se utilizarán las bibliotecas oficiales cuBLAS y cuDNN, plantillas de bajo nivel mediante CUTLASS, y la API WMMA para los kernels personalizados.

La recolección de métricas abarcará el throughput computacional medido en TFLOPS y la latencia de ejecución. De manera simultánea, se cuantificará el consumo energético medido físicamente a través de NVML y RAPL, y se realizará el análisis de degradación matemática utilizando las normas L_2 y L_∞ para cuantificar el drift acumulado.

5.5.2 Limitaciones

Para garantizar la viabilidad del estudio y enfocar el esfuerzo de ingeniería en la interacción entre la aritmética computacional y el consumo de potencia, se establecen restricciones claras sobre el entorno experimental. En primer lugar, se excluye explícitamente el análisis de cuantización en formatos enteros (como INT8 o INT4) y formatos de ultra-baja precisión (como FP8). Estos formatos poseen un rango dinámico matemáticamente insuficiente para la representación de variables físicas en simulaciones continuas, relegando su uso exclusivo al ámbito de la inferencia en Inteligencia Artificial.

Desde la perspectiva arquitectónica, la investigación está estrictamente limitada al hardware de NVIDIA. No se evaluará la portabilidad a aceleradores gráficos de otros fabricantes, como las matrices Matrix Cores de AMD CDNA o las extensiones XMX de Intel. Esta exclusión obedece a que el manejo explícito de la precisión mixta requiere invocaciones a bajo nivel que son altamente dependientes del conjunto de instrucciones de cada microarquitectura.

Asimismo, se excluye la ejecución distribuida en entornos Multi-GPU o clústeres basados en MPI. El proyecto se enfocará en aislar el comportamiento termodinámico y numérico dentro de una única GPU. Esta decisión busca evitar que la latencia y el consumo de energía inherentes a las interfaces de red, como InfiniBand, introduzcan variables de confusión en el cálculo del Producto Energía-Retardo (EDP).

Finalmente, el alcance del estudio no incluye el desarrollo de heurísticas de auto-ajuste dinámico que modifiquen la precisión en tiempo real durante la ejecución. El análisis se restringirá a una evaluación estática y comparativa, realizada a posteriori, de las configuraciones paramétricas preestablecidas.

6 CRONOGRAMA

El plan de ejecución se ha diseñado para un horizonte temporal de 16 semanas, estructurado en cuatro fases consecutivas que garantizan la validación incremental de los resultados numéricos y de rendimiento. La distribución temporal prioriza el

establecimiento de un Ground Truth robusto en la etapa inicial para mitigar riesgos en las fases posteriores de evaluación de pérdida de precisión.

Actividades	Mes 1				Mes 2				Mes 3				Mes 4			
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
FASE 1: Construcción de la Línea Base Analítica (FP64/FP32)																
Fundamentación de formatos FP64/FP32 y configuración del entorno CUDA																
Implementación de GEMM, Convolución 2D y Stencil 2D en FP64/FP32																
Validación numérica y línea base optimizada con CPU, cuBLAS y cuDNN																
Perfilamiento del rendimiento base con Nsight Compute																
FASE 2: Integración de Precisión Mixta y Tensor Cores																
Refactorización de GEMM y Convolución 2D a FP16/BF16 con cuBLAS/cuDNN																
Optimización de acceso a memoria y coalescencia a nivel de Warp																
Implementación de kernels WMMA utilizando Tensor Cores																
FASE 3: Cuantificación de Drift y Suma Compensada																
Implementación de kernels custom de Stencil 2D en formatos de precisión reducida																
Inserción del algoritmo de compensación numérica de Kahan																
Evaluación del error numérico (normas L2 y L∞) respecto a FP64																
FASE 4: Telemetría, Diseño Factorial y Análisis																
Barrido paramétrico y medición de rendimiento y consumo energético (NVML/RAPL)																
Análisis estadístico (t-test/ANOVA) y modelado del Frente de Pareto																
FASE 5: Documentación																
Redacción continua del manuscrito científico (IMRAD)																
Elaboración de figuras finales y presentación de resultados experimentales																

7 PRESUPUESTO

Relacione los principales gastos del proyecto y la persona o entidad que cubrirá esos gastos.

- Recursos humanos: asesoría profesor (tener en cuenta el valor indicado por la Vicerrectoría de Investigación y Extensión), estudiantes, etc.
- Equipo: computador, costos de uso de servidores, servicios web, etc.
- Materiales: impresiones, fotocopias, etc.
- Otros: salidas de campos, transporte, etc.

La estimación presupuestal obedece a los costos de referencia institucional para proyectos de investigación. La valoración se presenta bajo la modalidad de aporte en

especie (in-kind), diferenciando la carga operativa humana (desarrollo de software) de la carga computacional (validación experimental).

Recursos Humanos

CARGO	NOMBRE	DEDICACIÓN	VALOR UNITARIO (Estimado)	TOTAL
Director	Gilberto Javier Díaz Toro	64 horas	$41.000	$2.624.000
Estudiante Investigador	Karen Dayana Mateus Gomez	320 horas	$10.000	$3.200.000
Estudiante Investigador	William Andrés Urrutia Torres	320 horas	$10.000	$3.200.000
SUBTOTAL				**$7.024.000**

Recursos Tecnológicos e Infraestructura

DESCRIPCIÓN	UNIDAD	VALOR UNITARIO	TOTAL
Uso de Supercomputadora (SC3)	5.000 horas-núcleo*	$9.000	$45.000.000
Computador Personal	2 Unidades	$5.000.000	$10.000.000
SUBTOTAL			**$55.000.000**

* **Nota Aclaratoria**: La unidad "hora-núcleo" mide esfuerzo de procesamiento paralelo, no tiempo de calendario. Dado que cada experimento utiliza simultáneamente los 32 núcleos del nodo computacional, cada hora real consume 32 horas del presupuesto. Por tanto, las 5.000 horas-núcleo solicitadas equivalen a solo 156 horas reales (Wall-time) de ocupación del equipo ($5.000 \div 32 \approx 156$), lo cual es viable dentro del cronograma académico.

Total General

Métrica	Valor	Descripción
Esfuerzo Computacional Total	5,000 horas-núcleo	Carga acumulada por la ejecución de la matriz experimental completa (Stencils, GEMM y Convoluciones evaluados en FP64, FP32, FP16 y BF16).
Recursos Solicitados	1 Nodo (32 cores + GPU)	Capacidad de procesamiento híbrido requerida para la aceleración mediante *Tensor Cores* y telemetría termodinámica.
Factor de Paralelismo	32x	Tasa de aceleración promedio y reserva equivalente por el uso concurrente de los recursos del nodo.
Tiempo Real Estimado	~156 horas (6.5 días)	Tiempo físico requerido para completar las validaciones de *throughput*, energía (EDP) y error numérico.

8 BIBLIOGRAFÍA

[1] A. Kashi et al., “Mixed-precision numerics in scientific applications: survey and perspectives,” arXiv (Cornell University), Dec. 2024, doi: 10.48550/arxiv.2412.19322.

[2] E. Suarez et al., “Energy efficiency trends in HPC: what high-energy and astrophysicists need to know,” Frontiers in Physics, vol. 13, Apr. 2025, doi: 10.3389/fphy.2025.1542474.

[3] M. Andersch and M. Andersch, “NVIDIA Hopper Architecture In-Depth,” NVIDIA Technical Blog, Jul. 22, 2025. [Online]. Available: https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/

[4] F. A. Khattak and M. Mikaitis, “Accurate models of NVIDIA tensor cores,” arXiv (Cornell University), Dec. 2025, doi: 10.48550/arxiv.2512.07004.

[5] E. Atoofian, “PTTS: Power-aware tensor cores using two-sided sparsity,” Journal of Parallel and Distributed Computing, vol. 173, pp. 70–82, Mar. 2023, doi: 10.1016/j.jpdc.2022.11.004.

[6] Y. Chen, K. Li, Y. Wang, D. Bai, L. Wang, L. Ma, L. Yuan, Y. Zhang, T. Cao, and M. Yang, “ConvStencil: Transform Stencil Computation to Matrix Multiplication on Tensor Cores,” in Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP ’24), Edinburgh, United Kingdom, Mar. 2024, doi: 10.1145/3627535.3638476.

[7] A. Abdelfattah, H. Anzt, E. G. Boman, J. J. Dongarra, H. F. Fathololoumi, A. Haidar, G. Heuveline, M. M. A. Hussein, H. Imachi, P. Lal Awasthi, A. Tomov, and S. Yamazaki, “A survey of numerical linear algebra methods utilizing mixed-precision arithmetic,” The International Journal of High Performance Computing Applications, vol. 35, no. 4, pp. 344–369, 2021, doi: 10.1177/10943420211003313.

[8] P. Blanchard, N. J. Higham, F. Lopez, T. Mary, and S. Pranesh, “Mixed Precision Block Fused Multiply-Add: Error Analysis and Application to GPU Tensor Cores,” SIAM Journal on Scientific Computing, vol. 42, no. 3, pp. C124–C141, 2020, doi: 10.1137/19M1289546.

[9] A. Haidar, H. Bayraktar, S. Tomov, J. Dongarra, and N. J. Higham, “Mixed-precision iterative refinement using tensor cores on GPUs to accelerate solution of linear systems,” Proceedings of the Royal Society A, vol. 476, no. 2243, 20200110, 2020, doi: 10.1098/rspa.2020.0110.

[10] X.-Y. Liu, H. Hong, Z. Zhang, W. Tong, J. Kossaifi, X. Wang, and A. Walid, “High-Performance Tensor-Train Primitives Using GPU Tensor Cores,” *IEEE Transactions on Computers*, vol. 73, no. 11, pp. 2634–2642, Nov. 2024, doi: 10.1109/TC.2024.3441831.

[11] E.-M. El Arar, S.-I. Filip, T. Mary, and E. Riccietti, “Mixed precision accumulation for neural network inference guided by componentwise forward error analysis,” arXiv (Cornell University), Dec. 2025.

[12] A. Netti, Y. Peng, P. Omland, M. Paulitsch, J. Parra, G. Espinosa, U. Agarwal, A. Chan and K. Pattabiraman, “Mixed precision support in HPC applications: What about reliability?,” Journal of Parallel and Distributed Computing, vol. 181, art. 104746, 2023, doi: 10.1016/j.jpdc.2023.104746.

[13] A. Haidar, S. Tomov, J. Dongarra and N. J. Higham, “Harnessing GPU Tensor Cores for Fast FP16 Arithmetic to Speed up Mixed-Precision Iterative Refinement Solvers,” in SC18: International Conference for High Performance Computing, Networking, Storage and Analysis, 2018, doi: 10.1109/SC.2018.00050.

[14] Y. Chen, P. de Oliveira Castro, P. Bientinesi, N. Jansson and R. Iakymchuk, “Enabling mixed-precision in spectral element codes,” Future Generation Computer Systems, vol. 174, art. 107990, 2026, doi: 10.1016/j.future.2025.107990.

[15] M. Fasi, N. J. Higham, M. Mikaitis and S. Pranesh, “Numerical behavior of NVIDIA tensor cores,” *PeerJ Computer Science*, vol. 7, p. e330, Feb. 2021. doi: 10.7717/peerj-cs.330.

[16] P. M. Basso, F. F. dos Santos and P. Rech, “Impact of Tensor Cores and Mixed Precision on the Reliability of Matrix Multiplication in GPUs,” *IEEE Transactions on Nuclear Science*, vol. 67, no. 7, pp. 1560–1565, Jul. 2020. doi: 10.1109/TNS.2020.2977583.

[17] M. Fasi, N. J. Higham, F. Lopez, T. Mary and M. Mikaitis, “Matrix Multiplication in Multiword Arithmetic: Error Analysis and Application to GPU Tensor Cores,” *SIAM Journal on Scientific Computing*, vol. 45, no. 1, pp. C1–C19, 2023. doi: 10.1137/21M1465032.

[18] M. Hafezan and E. Atoofian, “Mixed-Precision Architecture for GPU Tensor Cores,” in *Proceedings of the 23rd IEEE International Conference on Scalable Computing and Communications (ScalCom)*, 2023, pp. 1–8. doi: 10.1109/SWC57546.2023.10448789.

[19] M. Baboulin, S. Donfack, O. Kaya, T. Mary and M. Robeyns, “Mixed precision randomized low-rank approximation with GPU tensor cores,” in *Euro-Par 2024: Parallel Processing*, pp. 31–44, 2024. doi: 10.1007/978-3-031-69583-4_3.